

Documentation: 3D Drone Rescue Navigation

A* Search Algorithm for Smart City Rescue Mission
AI Lab — Assignment 4

Uttam Mahata (2022CSB104)

March 10, 2026

Contents

1 Introduction

A disaster has struck a futuristic smart city. An AI-powered drone must autonomously navigate through the city, avoiding collapsed buildings and fire zones, rescue all stranded survivors, and return to base — while minimising total energy consumption.

This implementation uses the **A* search algorithm** extended to a three-dimensional grid. The drone operates in a layered city space (Ground, Mid, and Sky levels) and must visit multiple intermediate goal states (survivor locations) before reaching the final goal (base station).

Key challenges addressed:

- 3D state-space navigation with heterogeneous movement costs.
- Multi-goal search using greedy nearest-neighbour ordering.
- Dynamic recharge logic when energy becomes insufficient mid-mission.
- Admissible 3D Euclidean heuristic that guarantees optimal paths per segment.

2 Problem Formulation

2.1 State Space

The smart city is modelled as a 3-dimensional grid. Each cell is identified by coordinates (x, y, z) where:

- $x \in \{0, \dots, \text{rows} - 1\}$ — row index (North–South axis).
- $y \in \{0, \dots, \text{cols} - 1\}$ — column index (East–West axis).
- $z \in \{0, \dots, \text{levels} - 1\}$ — altitude level (0 = Ground, 1 = Mid, 2 = Sky).

2.2 Cell Types

Symbol	Type	Description
0	Road / Open space	Passable; base terrain cost = 1
1	Building / Obstacle	Impassable; drone cannot pass
F	Fire zone	Passable; high-risk; terrain cost = 3
S	Drone start	Passable (cost = 1); mission origin
Gn	Survivor location	Passable (cost = 1); must be visited
R	Recharging station	Passable (cost = 1); restores energy
B	Base station	Passable (cost = 1); final goal

Table 1: Cell types and their properties.

2.3 Initial and Goal States

- **Initial state:** the drone is at position S with energy E_0 .

- **Intermediate goals:** all survivor locations $G_1, G_2, \dots, G_n$ must be visited (in any order).
- **Final goal:** the drone must return to base B after rescuing all survivors.

3 Movement Model

3.1 Six-Directional Movement

The drone moves in six axis-aligned directions — no diagonal movement is permitted:

Direction	Coordinate change
North	$(x - 1, y, z)$
South	$(x + 1, y, z)$
East	$(x, y + 1, z)$
West	$(x, y - 1, z)$
Up	$(x, y, z + 1)$
Down	$(x, y, z - 1)$

3.2 Energy Cost Model

The energy cost $c(u, v)$ to move from cell u to adjacent cell v depends on both the *terrain* of the destination cell and the *direction* of movement:

$$c(u, v) = \begin{cases} \tau(v) & \text{if horizontal move } (\Delta z = 0) \\ \tau(v) + 2 & \text{if ascending } (\Delta z = +1) \\ 1 & \text{if descending } (\Delta z = -1) \end{cases} \quad (1)$$

where the terrain cost $\tau(v)$ is:

$$\tau(v) = \begin{cases} 3 & \text{if } v \text{ is a fire zone (F)} \\ 1 & \text{otherwise (open space, S, B, R, or } G_n) \end{cases} \quad (2)$$

Rationale:

- Horizontal flight through free space costs 1 unit.
- Fire zones create turbulence — horizontal traversal costs 3 units.
- Ascending requires extra thrust against gravity: +2 energy units on top of the terrain cost.
- Descending exploits gravity assist and always costs just 1 unit, regardless of terrain below.

4 A* Algorithm

4.1 Evaluation Function

A* evaluates each node n using the combined function:

$$f(n) = g(n) + h(n) \quad (3)$$

- $g(n)$ — actual energy consumed from the start to node n .
- $h(n)$ — admissible heuristic estimate of the energy from n to the goal.
- $f(n)$ — estimated total energy for the best path through n .

4.2 Heuristic Function

The heuristic uses the **3D Euclidean distance**:

$$h(n) = \sqrt{(x_n - x_g)^2 + (y_n - y_g)^2 + (z_n - z_g)^2} \quad (4)$$

Admissibility: Since the minimum possible energy for any single step is 1 unit (descending through free space), and the Euclidean distance never overestimates the number of steps required ($h_{\text{Euclidean}} \leq d_{L1}$), we have:

$$h(n) \leq h^*(n) \quad \forall n \quad (5)$$

where $h^*(n)$ is the true minimum energy cost to the goal. This guarantees A* finds the *optimal* (minimum energy) path per segment.

4.3 Priority Queue

Nodes are stored in a **min-heap** (priority queue) ordered by $f(n)$. A tie-breaking counter ensures that no two heap entries are compared on their position tuples:

```
1 heapq.heappush(open_set, (f_score, g_score, next(counter),  
    position))
```

Listing 1: Priority queue push with tie-breaking.

Stale (outdated) entries are lazily discarded:

```
1 if g_val > g_score.get(current, float('inf')):  
2     continue # outdated entry skip
```

Listing 2: Skipping outdated entries.

4.4 Algorithm Pseudocode

```
1 Input:  city grid G, start s, goal t  
2 Output: shortest path P and cost c, or None if unreachable  
3  
4 open  <- { (h(s,t), 0, s) }          -- min-heap: (f, g, node)  
5 g[s]  <- 0 ; g[v] <- inf for all v != s
```

```

6  came_from <- {}
7
8  while open != empty:
9      (f, g_val, u) <- open.pop_min()
10     if g_val > g[u]: continue          -- stale entry
11     if u = t: return reconstruct(came_from, u), g[t]
12     for each neighbour v of u:
13         g' <- g[u] + c(u, v)
14         if g' < g[v]:
15             g[v] <- g'
16             came_from[v] <- u
17             open.push( g' + h(v,t), g', v )
18
19 return None, inf

```

Listing 3: A* on 3D City Grid (pseudocode)

5 Multi-Goal Search Strategy

5.1 Segment-by-Segment A*

The full mission is decomposed into independent A* searches:

$$S \xrightarrow{A^*} G_{\pi(1)} \xrightarrow{A^*} G_{\pi(2)} \xrightarrow{A^*} \dots \xrightarrow{A^*} G_{\pi(n)} \xrightarrow{A^*} B$$

where π is the visit permutation determined by the goal ordering heuristic.

5.2 Greedy Nearest-Neighbour Goal Ordering

Finding the optimal visit order for n survivors is equivalent to the Travelling Salesman Problem (NP-hard). For practical sizes, a **greedy nearest-neighbour** heuristic is applied:

```

1 Input:  start s,  goal set G
2 Output: ordered list of goals
3
4 ordered <- [] ;  current <- s ;  R <- G
5 while R != empty:
6     g* <- argmin_{g in R} h(current, g)
7     ordered.append(g*)
8     current <- g*
9     R <- R \ {g*}
10 return ordered

```

Listing 4: Greedy Nearest-Neighbour Goal Ordering (pseudocode)

The Euclidean distance h is used for proximity estimation.

6 Recharge Logic

6.1 Energy Tracking and Recharge Trigger

The drone tracks remaining energy E throughout the mission. Before each segment, the A* cost c_{seg} to the next goal is computed. If $c_{\text{seg}} > E$, the recharge procedure is triggered:

1. Find the nearest reachable recharging station R^* via A*.
2. If $\text{cost}(R^*) > E$: **Mission failure** — energy depleted before reaching any station.
3. Fly to R^* , deducting travel cost from E .
4. **Recharge:** $E \leftarrow E_0 - p$, where E_0 is the initial energy and p is the configurable recharge penalty.
5. Re-run A* from R^* to the original target goal.

6.2 Recharge Penalty

Stopping at a recharge station incurs a fixed energy penalty p (default: 5 units) to model the time cost of landing, docking, and recharging. After recharging, the drone has energy $E_0 - p$ available (full recharge minus the penalty), ensuring the penalty discourages unnecessary detours.

7 Implementation

7.1 Module Structure

```
1 drone_navigation.py
2     Constants (OBSTACLE, FIRE, DIRECTIONS, LEVEL_NAMES)
3     CityGrid # loads and queries the 3D
4     grid
5     __init__(data) # parse JSON, locate S/B/
6     Gn/R
7     get_cell(x, y, z) # bounds-safe cell lookup
8     terrain_cost(cell) # 3 for F, 1 otherwise
9     is_passable(cell) # False for obstacle '1'
10    move_cost(dest, dz) # direction-aware energy
11    cost
12    get_neighbors(pos) # valid (neighbour, cost)
13    pairs
14    heuristic(a, b) # 3D Euclidean distance
15    astar_3d(grid, start, goal) # core A* returns (path
16    , cost)
17    _reconstruct_path(...) # backtrack came_from
18    _greedy_goal_order(...) # nearest-neighbour TSP
19    heuristic
20    _find_nearest_recharge(...) # A* to closest R station
21    plan_mission(...) # orchestrates full mission
```

```

16         visualize_grid(...)                # ASCII layer-by-layer
output
17         main()                             # CLI entry point

```

Listing 5: Module-level layout of drone_navigation.py

7.2 CityGrid Class

The CityGrid class wraps the JSON grid data and provides neighbour-lookup with full cost computation:

```

1 def get_neighbors(self, pos):
2     x, y, z = pos
3     result = []
4     for dx, dy, dz, _ in DIRECTIONS:
5         nx, ny, nz = x + dx, y + dy, z + dz
6         cell = self.get_cell(nx, ny, nz)
7         if self.is_passable(cell):
8             result.append(((nx, ny, nz), self.move_cost(cell, dz)
9     ))
10    return result

```

Listing 6: CityGrid.get_neighbors — cost-aware 3D neighbourhood.

7.3 A* Core

The astar_3d function uses a min-heap with lazy deletion of stale entries:

```

1 while open_set:
2     f_val, g_val, _, current = heapq.heappop(open_set)
3
4     if current == goal:
5         return _reconstruct_path(came_from, current), g_score[
goal]
6
7     if g_val > g_score.get(current, float('inf')):
8         continue # stale entry
9
10    for neighbor, cost in city_grid.get_neighbors(current):
11        tentative = g_score[current] + cost
12        if tentative < g_score.get(neighbor, float('inf')):
13            came_from[neighbor] = current
14            g_score[neighbor] = tentative
15            f_new = tentative + heuristic(neighbor, goal)
16            heapq.heappush(open_set,
17                            (f_new, tentative, next(_TIE_COUNTER),
neighbor))

```

Listing 7: Core A* loop.

7.4 Input Format

The city grid is provided as a **JSON file** with the following schema:

```

1 {
2   "dimensions": { "rows": 5, "cols": 5, "levels": 3 },
3   "initial_energy": 100,
4   "recharge_penalty": 5,
5   "grid": [
6     [ /* z=0 Ground Level */ [...], [...], ... ],
7     [ /* z=1 Mid Level      */ [...], [...], ... ],
8     [ /* z=2 Sky Level      */ [...], [...], ... ]
9   ]
10 }
```

Listing 8: city_grid.json structure.

7.5 Command-Line Interface

```

1 python drone_navigation.py city_grid.json
2 python drone_navigation.py city_grid.json --energy 80 --recharge-
   penalty 10
```

Listing 9: Usage.

Argument	Default	Description
grid_file	—	Path to city grid JSON
--energy	from JSON (100)	Initial drone energy
--recharge-penalty	from JSON (5)	Energy penalty for recharging

8 Example: 5×5×3 City Grid

8.1 Grid Layout

The example grid has 3 levels ($z = 0, 1, 2$) each of size 5×5 .

Ground Level ($z = 0$):

S	0	1	0	0
0	F	1	1	G_1
F	0	0	0	F
R	1	1	1	0
B	0	1	0	G_2

Mid Level ($z = 1$):

0	0	1	0	0
0	1	1	0	0
F	0	0	0	1
0	1	1	1	0
0	0	0	0	0

Sky Level ($z = 2$):

1	0	0	0	0
0	<i>F</i>	0	1	0
0	0	1	0	<i>F</i>
0	0	0	0	0
0	1	0	0	1

8.2 Special Positions

Entity	Position (x, y, z)
Start S	(0, 0, 0)
Survivor G_1	(1, 4, 0)
Survivor G_2	(4, 4, 0)
Recharge R	(3, 0, 0)
Base B	(4, 0, 0)

9 Edge Cases

1. **No valid path to a survivor:** A^* returns `None` for that segment. The mission halts with an error message.
2. **Energy depleted before reaching a recharge station:** If $\text{cost}(R^*) > E$, the drone cannot reach any station. The mission is declared failed.
3. **Base station unreachable:** A^* on the final segment returns `None`. Error reported; partial path and consumed energy are still output.
4. **No survivor goals in grid:** The drone flies directly from S to B in a single A^* segment.
5. **No recharge stations in grid:** If energy runs out mid-mission, mission fails with a clear error rather than an infinite loop.
6. **Start equals goal:** `astar_3d` short-circuits and returns `([start], 0.0)`.

10 Complexity Analysis

Let $N = \text{rows} \times \text{cols} \times \text{levels}$ be the total number of cells and k be the number of survivors.

Component	Time	Space
Single A^* call	$O(N \log N)$	$O(N)$
Goal ordering (greedy)	$O(k^2)$	$O(k)$
Find nearest recharge	$O(\mathcal{R} \cdot N \log N)$	$O(N)$
Full mission ($k + 1$ segments)	$O(k \cdot N \log N)$	$O(N)$

Table 2: Asymptotic complexity ($\mathcal{R}$ = set of recharge stations).

The space complexity per A^* call is $O(N)$ for the `g_score` dict and the `came_from` backtracking map.

11 Conclusion

This implementation extends the classical A* algorithm to a fully 3D, heterogeneous-cost, multi-goal rescue scenario. Key design decisions:

- **Admissible 3D heuristic** — 3D Euclidean distance guarantees optimality for each individual segment.
- **Direction-aware costs** — ascending and descending have physically motivated asymmetric energy penalties.
- **Greedy goal ordering** — practical $O(k^2)$ nearest-neighbour heuristic avoids NP-hard TSP exact search.
- **Lazy recharge** — recharging is only triggered when necessary, minimising the fixed penalty cost.
- **Configurable parameters** — initial energy and recharge penalty can be specified via JSON or CLI flags, enabling scenario testing.